

Kubernetes – Prérequis & Fondamentaux

Jour 2 — Fondamentaux Kubernetes

Partie 1 : architecture, objets de base, exposition interne

Kubernetes — Prérequis & Fondamentaux — m2i
Module 2/3

Objectifs du module

À la fin de cette journée, vous serez capable de :

- Justifier l'apport de Kubernetes par rapport à Docker seul
- Schématiser l'architecture d'un cluster (control plane + nœuds)
- Manipuler kubectl avec les commandes essentielles
- Créer et opérer des Pods, ReplicaSets et Deployments
- Exposer un service en interne via ClusterIP, NodePort, LoadBalancer
- Diagnostiquer une application qui ne démarre pas

Durée : 7 heures (1 journée)

Plan

1. Pourquoi Kubernetes ?
2. Architecture d'un cluster
3. kubectl et objets de base
4. Pods : l'unité de base
5. Workloads : ReplicaSet et Deployment
6. Services : exposition réseau interne
7. Démonos et TP

1. Pourquoi Kubernetes ?

Les limites de Docker seul

Ce que Docker fait bien

- Construire et exécuter des conteneurs
- Compose pour une stack locale
- Réseau et volumes basiques

Ce qui manque en prod

- Réplication multi-nœuds
- Auto-réparation et reschedule
- Découverte de service à l'échelle
- Mises à jour sans interruption
- Gestion déclarative et versionnée

Positionnement de Kubernetes

“ Kubernetes est un orchestrateur déclaratif qui réconcilie un état désiré et un état réel. ”
Trois mots clés :

- **Déclaratif** : vous décrivez ce que vous voulez, pas comment l'obtenir
- **Réconciliation** : des contrôleurs ramènent le système vers l'état désiré en continu
- **Auto-réparation** : un Pod qui crash est redémarré ; un nœud qui tombe voit ses Pods rescheduled

Écosystème CNCF

```
+----- CNCF Landscape -----+
| Orchestration : Kubernetes (graduated) |
| Container runtime : containerd, CRI-O   |
| Réseau : Cilium, Calico, Flannel        |
| Observabilité : Prometheus, OpenTelemetry, Jaeger |
| Sécurité : Falco, OPA, Cert-manager     |
| Stockage : Rook, Longhorn, OpenEBS      |
+-----+
```

Distributions : vanilla, managées (EKS, GKE, AKS), on-premises (k0s, k3s, OpenShift).

2. Architecture d'un cluster

Vue d'ensemble

```

+----- Control plane -----+
| kube-apiserver  <- entrée unique (REST + auth) |
| etcd           <- stockage clé-valeur de l'état |
| kube-scheduler <- choisit le nœud pour chaque Pod |
| kube-controller-manager <- boucles de contrôleurs |
+-----+
|
| +-----+
| v               v
+----- Worker 1 -----+ +--- Worker 2 ---+
| kubelet  -> dialogue avec apiserver | | ... |
| kube-proxy -> règles iptables/IPVS | | |
| containerd -> exécute les conteneurs | | |
+-----+ +-----+

```

Le control plane

Composant	Rôle
kube-apiserver	Point d'entrée REST, authentification, validation
etcd	Base de données clé-valeur distribuée (état du cluster)
kube-scheduler	Affecte un nœud à chaque Pod non assigné
kube-controller-manager	Boucles : Node, Replication, Endpoints, etc.
cloud-controller-manager	Spécifique aux providers cloud

`etcd` est le composant le plus sensible : à sauvegarder régulièrement.

Les nœuds (workers)

Composant	Rôle
kubelet	Agent qui lance les conteneurs demandés par l'apiserver
kube-proxy	Maintient les règles réseau pour les Services
container runtime	containerd (par défaut) ou CRI-O

```
kubectl get nodes -o wide  
kubectl describe node kind-worker
```

Flux d'une requête `kubectl apply`

1. `kubectl apply -f deployment.yaml`
2. kube-apiserver authentifie et valide
3. kube-apiserver écrit l'objet Deployment dans etcd
4. deployment-controller observe et crée un ReplicaSet
5. replicaset-controller crée N Pods (état "Pending")
6. kube-scheduler choisit un nœud pour chaque Pod
7. kubelet du nœud lance le conteneur via containerd
8. kubelet met à jour le statut dans apiserver

Trois boucles de contrôle séquentielles, pas une instruction impérative.

Kind : cluster local en v1.34

```
cat > kind-config.yaml <<EOF
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
EOF

kind create cluster --name m2i --image kindest/node:v1.34.0 --config kind-config.yaml
kubectl cluster-info --context kind-m2i
kubectl get nodes
```

kind lance chaque nœud dans un conteneur Docker. Multi-nœuds = isolation des réseaux.

3. kubectl et objets de base

Anatomie d'un manifeste YAML

```
apiVersion: apps/v1          # quel groupe/version d'API
kind: Deployment             # quel type d'objet
metadata:
  name: web
  namespace: prod
  labels:
    app: web
spec:                         # état désiré
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
        - name: web
          image: nginx:1.27
```


Les 10 commandes kubectl à connaître

```
kubectl get pods -A           # tous namespaces
kubectl get deploy -n prod -o wide # détaillé
kubectl describe pod web-abc123  # diagnostic
kubectl apply -f manifest.yaml   # créer ou mettre à jour
kubectl delete -f manifest.yaml  # supprimer
kubectl logs -f web-abc123 -c sidecar # logs en suivi
kubectl exec -it web-abc123 -- sh  # shell dans le conteneur
kubectl port-forward svc/web 8080:80 # tunnel local
kubectl rollout status deploy/web  # statut d'un rollout
kubectl explain pod.spec.containers # documentation embarquée
```

`kubectl explain` est sous-utilisé : c'est la doc de référence locale.

Namespaces

“ Un namespace est une frontière d'isolation logique pour nommer les ressources. ”

```
kubectl create namespace dev  
kubectl get all -n dev  
kubectl config set-context --current --namespace=dev
```

Cas d'usage :

- Séparer dev / staging / prod
- Multi-tenant léger (un namespace par équipe)
- Quotas et limites par namespace

Les Nodes, PersistentVolumes et StorageClasses sont cluster-scoped, pas namespacés.

kubeconfig, contextes et clusters

```
kubectl config view --raw          # voir
kubectl config get-contexts        # lister
kubectl config use-context kind-m2i # basculer
kubectl config set-context --current --namespace=prod # ns par défaut
```

Un `kubeconfig` regroupe : `clusters`, `users`, `contexts`. Un contexte = un triplet (cluster, user, namespace).

Productivité kubectl

```
alias k=kubectl
source <(kubectl completion bash)
complete -F __start_kubectl k

# kubectx et kubens pour basculer rapidement
kubectx kind-m2i
kubens prod

# k9s pour une vue tableau temps réel
k9s
```

Gain mesuré : ~30% du temps debug économisé avec l'autocomplétion et k9s.

4. Pods : l'unité de base

Qu'est-ce qu'un Pod ?

“ Un Pod est le plus petit objet déployable : 1 ou N conteneurs partageant réseau et stockage. ”

```
apiVersion: v1
kind: Pod
metadata: {name: web}
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports: [{containerPort: 80}]
```

Tous les conteneurs d'un Pod :

- Partagent l'IP du Pod (`localhost` entre eux)
- Peuvent partager des volumes
- Sont co-schedulés sur le même nœud

Pod multi-conteneurs : patterns

Sidecar

- Logger, proxy, sync de fichiers
- Vit aussi longtemps que le main
- Cas : Envoy, Fluent Bit

Adapter

- Normalise des sorties
- Ex : convertit logs maison en JSON

Ambassador

- Proxy sortant vers un service externe
- Ex : proxy SQL local

À retenir : un Pod = une unité fonctionnelle, pas une collection arbitraire.

Cycle de vie d'un Pod

État	Signification
Pending	Pod accepté, pas encore démarré (image en pull, scheduling)
Running	Au moins un conteneur tourne
Succeeded	Tous les conteneurs terminés avec succès (Jobs)
Failed	Au moins un conteneur a échoué
Unknown	Communication perdue avec le nœud

```
kubectl get pod web -o jsonpath='{.status.phase}'  
kubectl get events --sort-by=.lastTimestamp
```


Sondes de santé

```
spec:
  containers:
    - name: app
      image: app:1.0
      livenessProbe:           # redémarrer si KO
        httpGet: {path: /healthz, port: 8080}
        initialDelaySeconds: 15
        periodSeconds: 10
      readinessProbe:         # retirer du Service si KO
        httpGet: {path: /ready, port: 8080}
      startupProbe:           # gros warm-up Java
        httpGet: {path: /healthz, port: 8080}
        failureThreshold: 30
        periodSeconds: 10
```

À retenir : Spring Boot Actuator expose `/actuator/health/liveness` et `/readiness` clés en main.

Requests et limits

```
spec:
  containers:
    - name: app
      image: app:1.0
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
        limits:
          cpu: "1000m"
          memory: "512Mi"
```

- **requests** : ce que le scheduler garantit (réservation)
- **limits** : plafond. Dépasse CPU = throttling, dépasser mémoire = OOMKill

Recommandation : toujours définir les `requests` mémoire.

Init containers

```
spec:
  initContainers:
    - name: wait-db
      image: busybox:1.36
      command: ['sh', '-c', 'until nc -z db 5432; do sleep 2; done']
  containers:
    - name: app
      image: app:1.0
```

Cas d'usage : attendre une dépendance, migrer un schéma, télécharger un fichier de config.
Les init containers s'exécutent séquentiellement avant les conteneurs principaux.

5. ReplicaSet et Deployment

Du Pod au ReplicaSet

“ Un Pod brut n'est pas réparé en cas de crash. Un ReplicaSet maintient un nombre de répliques. ”

```
apiVersion: apps/v1
kind: ReplicaSet
metadata: {name: web-rs}
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers: [{name: web, image: nginx:1.27}]
```

En pratique, on ne crée plus de ReplicaSet directement : on passe par un Deployment.

Deployment : abstraction de référence

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  selector:
    matchLabels: {app: web}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers: [{name: web, image: nginx:1.27}]
```

Le Deployment gère un ReplicaSet par version d'image (historique).

RollingUpdate vs Recreate

RollingUpdate (défaut)

- Démarre les nouveaux Pods avant d'arrêter les anciens
- `maxSurge` : nb Pods en plus
- `maxUnavailable` : nb Pods coupés
- Zéro downtime

Par défaut : `maxSurge=25%`, `maxUnavailable=25%`.

Recreate

- Coupe tout l'ancien puis démarre le nouveau
- Cas : incompatibilité de schéma
- Downtime court mais réel

Rollout et historique

```
kubectl set image deploy/web web=nginx:1.28
kubectl rollout status deploy/web
kubectl rollout history deploy/web
kubectl rollout undo deploy/web
kubectl rollout undo deploy/web --to-revision=3
kubectl rollout pause deploy/web # canary manuel
kubectl rollout resume deploy/web
```

`kubectl rollout undo` est votre filet de sécurité.

Mise à l'échelle

```
kubectl scale deploy/web --replicas=5  
kubectl autoscale deploy/web --min=2 --max=10 --cpu-percent=70
```

Manuel = `scale`, automatique = HorizontalPodAutoscaler (vu dans le Module 2 suivant).

À retenir : `scale` n'est pas atomique au sens transactionnel. Observer avec `kubectl get pods -w`

.

6. Services : exposition réseau interne

Le problème résolu par le Service

“ Les Pods sont mortels et leurs IPs changent. Le Service offre une IP stable et un nom DNS. ”

```
Pod web-abc 10.244.1.5  +---> kill et reschedule ---+
Pod web-def 10.244.2.7  +                               v
Pod web-ghi 10.244.1.8                               web-jkl 10.244.3.2

Service web 10.96.42.10  (stable)
-> Endpoints mis à jour automatiquement
```

Selectors et endpoints

```
apiVersion: v1
kind: Service
metadata: {name: web}
spec:
  selector:
    app: web          # tous les Pods labellés app=web
  ports:
    - port: 80        # port du Service
      targetPort: 8080 # port du conteneur
```

```
kubectl get endpoints web
# NAME      ENDPOINTS
# web      10.244.1.5:8080,10.244.2.7:8080,10.244.1.8:8080
```

Types de Service

Internes

- **ClusterIP** (défaut) : IP virtuelle interne
- **Headless** (clusterIP: None) : pas d'IP, retour DNS multiples

Externes

- **NodePort** : port sur chaque nœud (30000-32767)
- **LoadBalancer** : provisionne un LB cloud
- **ExternalName** : alias DNS vers un FQDN

NodePort en prod = rarement la bonne réponse. Préférer Ingress (vu Module 3).

DNS interne (CoreDNS)

“ Chaque Service est résolvable via un nom DNS : `<service>.<namespace>.svc.cluster.local`

.

”

```
kubectl exec -it client -- nslookup web
# web.default.svc.cluster.local

kubectl exec -it client -- curl http://web.prod:80
```

Règles :

- Dans le même namespace : `web` suffit
- Vers un autre namespace : `web.prod`
- FQDN complet : `web.prod.svc.cluster.local`

Headless Services

```
apiVersion: v1
kind: Service
metadata: {name: db}
spec:
  clusterIP: None           # pas d'IP virtuelle
  selector: {app: postgres}
  ports: [{port: 5432}]
```

Cas d'usage : StatefulSet où chaque réplique a sa propre identité DNS.

db-0.db, db-1.db, db-2.db chacun sa propre adresse.

